

Status Report — Haskell File Search Tool (find/fd-like/ lamdaSearch)

Core directory traversal (mostly done)

- Uses POSIX directory streams via FFI (readdir + entry name/type), with EINTR retry and safe cleanup (finally).
- Recursively walks directories (treversRecursively), builds fullpath with (</>), filters files, and keeps traversing folders.

Works now: recursive traversal, DirContent for files/dirs, basic file filtering.

Limitations: single-threaded

2) Filtering (implemented)

Supported filters:

- Regex filter: regxPattern compiled via Text.Regex.TDFA (compileRegexFilter, getRexPattern)
- Extension filter: extention checked via takeExtension (maybe do that with regex)
- Hidden-file filtering: hideHidden with getHiddenFilter
- Exclude/ignore filter: exclude with getDisallowFilter

What works now:

- Filters are combined using and [rg, df, ef, hf] for each non-directory entry.
- Regex compilation is optional: if no pattern is provided, it matches everything.

What i should fix

- getDisallowFilter sf fp currently compares the **current directory path** (rfp) rather than the **candidate path** (fullpath) or directory entry path, which makes ignore rules behave unexpectedly.
- getHiddenFilter uses BC.head fp and can crash if fp is empty. (will fix)

3) CLI parsing (implemented for a custom command format)

Implemented module: ParseInput

- CLI input is parsed with Megaparsec.

Flag	Long Form	Purpose
-p	--pattern	SearchPatternFlag — Defines a search pattern to filter files.
-a	--show--dots	HiddenFilesFlag — Shows hidden files (those starting with a dot).
-e	--extention	ExtentionFlag — Filters files by extension.
-i	--ignore	IgnoreFlag — Specifies paths to exclude from processing.
-x	--execute	ExecuteFlag — Executes a command on processed files.

- Parsed output: SearchSetting { searchPaths, appliedCommand = Nothing, filters }

What works now:

- Parsing multiple quoted paths.
- Parsing multiple flags in flexible order.
- Producing a SearchSetting record.

Limitations:

- Execution flag is parsed but not wired into SearchSetting. (Not sure how i will end up doing this)
- Input handling is interactive (getLine) rather than traditional getArgs. (wil fix it)
- The command requires the literal prefix `lms` , which is fine for a school project but not typical UX. (will also fix)

Progress vs. original plan**Implemented (done or mostly done)**

- Core file-search logic.
- CLI parsing.
- Error handling approach via ExceptT + custom DirError (will make this better).
- Use of FFI / POSIX directory APIs.
- Basic filtering (regex, extension, hidden, ignore list).

Not implemented yet

- Parallel traversal / concurrency.
- Configuration file parsing and integration.
- Command execution on matched files.
- TUI (Brick).
- Fuzzy search functionality.

How to run

- Look at repo README
- <https://git.app.uib.no/martin.e.knutsen/inf221-Semesteroppgave.git>